

Wblt

WBLT

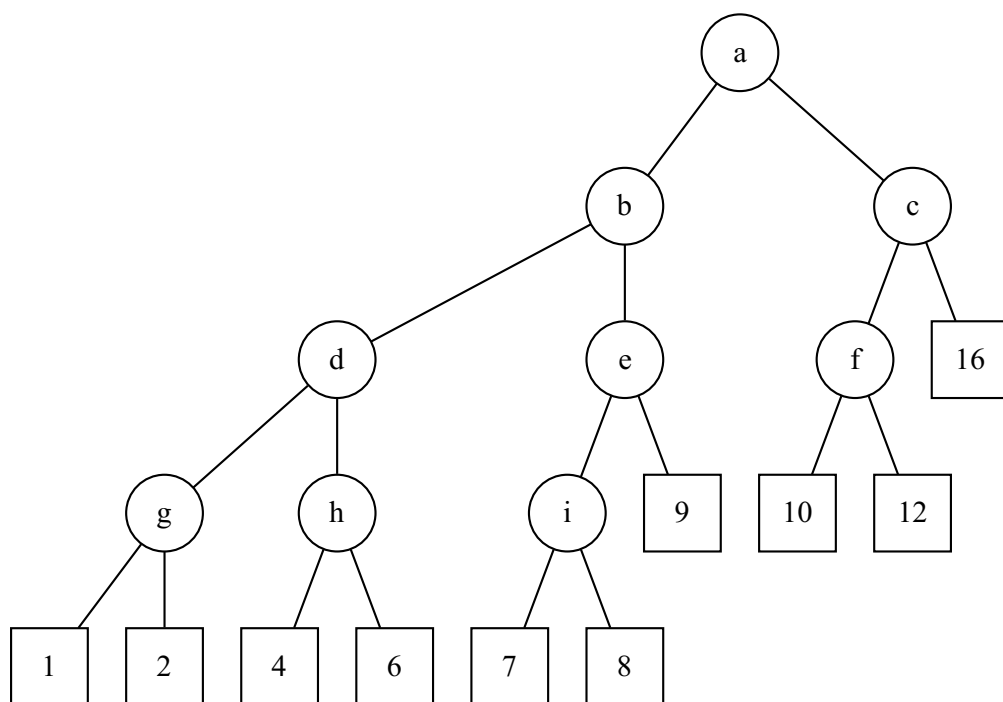
前言

Weight Balanced Leafy Tree, 下称 **WBLT**, 是一种平衡树, 比起其它平衡树主要有实现简单、常数小的优点。它支持区间操作, 而且可持久化。

Weight Balanced Leafy Tree 顾名思义是 Weight Balanced Tree 和 Leafy Tree 的结合。

Weight Balanced Tree 的每个结点储存这个结点下子树的大小, 并且通过保持左右子树的大小关系在一定范围来保证树高。

Leafy Tree 维护的原始信息仅存储在树的 **叶子节点** 上, 而非叶子节点仅用于维护子节点信息和维持数据结构的形态。我们熟知的线段树就是一种 Leafy Tree。



平衡树基础操作

代码约定

下文中, 我们用 $ls[x]$ 表示节点 x 的左儿子, $rs[x]$ 表示节点 x 的右儿子, $vl[x]$ 表示节点 x 的权值, $sz[x]$ 表示节点 x 及其子树中叶子节点的个数。

建树

正如前言中所说的, WBLT 的原始信息仅存储在叶子节点上。而我们规定每个非叶子节点一定有两个子节点, 这个节点要维护其子节点信息的合并。同时, 每个节点还要维护自身及其子树中叶子节点的数量, 用于实现维护平衡。

和大多数的平衡树一样, 每个非叶子节点的右儿子的权值大于等于左儿子的权值, 且在 WBLT 中非叶子节点节点的权值等于右儿子的权值。不难看出每个节点的权值就是其子树中的最大权值。

这样听起来就很像一棵维护区间最大值的动态开点线段树了，且所有叶子从左到右是递增的。事实上的建树操作也与线段树十分相似，只需要向下递归，直至区间长度为 1 时把要维护的信息放叶子节点上，回溯的时候合并区间信息即可。

代码实现如下：

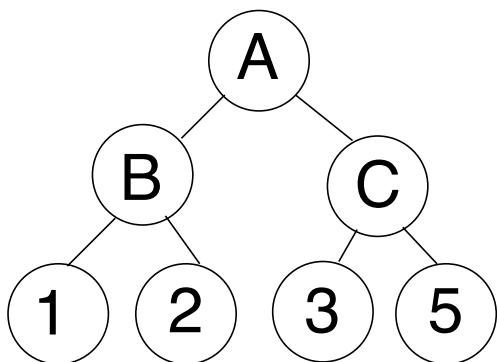
```
1 /* 添加一个权值为 v 的节点，返回这个节点的编号 */
2 int add(int v) {
3     ++cnt;
4     ls[cnt] = rs[cnt] = 0;
5     sz[cnt] = 1;
6     vl[cnt] = v;
7     return cnt;
8 }
9
10 /* 更新节点编号为 x 的节点的信息 */
11 void pushup(int x) {
12     vl[x] = vl[rs[x]];
13     sz[x] = sz[ls[x]] + sz[rs[x]];
14 }
15
16 /* 递归建树 */
17 int build(int l, int r) {
18     if (l == r) {
19         return add(a[l]);
20     }
21     int x = add(0);
22     int k = l + ((r - l) >> 1);
23     ls[x] = build(l, k);
24     rs[x] = build(k + 1, r);
25     pushup(x);
26 }
```

插入和删除

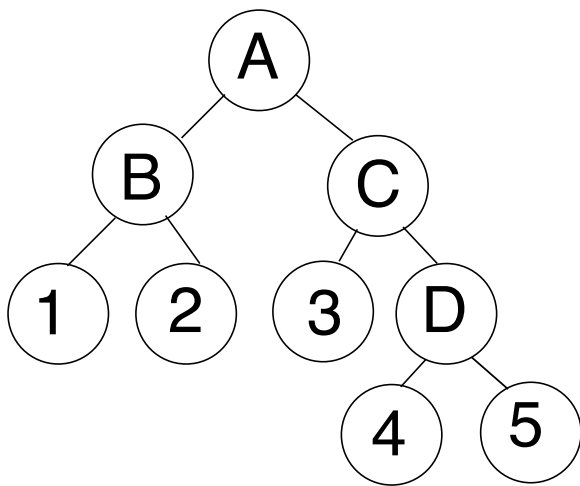
由于 WBLT 的信息都存储在叶子节点上，插入和删除一个元素其实就是增加或减少了一个叶子节点。

对于插入操作，我们类似从根节点开始向下递归，直到找到权值大于等于插入元素的权值最小的叶子节点，再新建两个节点，其中一个用来存储新插入的值，另一个作为两个叶子的新父亲替代这个最小叶子节点的位置，再将这两个叶子连接到这个父亲上。

例如我们向以下树中加入一个值为 4 的元素。



我们首先找到了叶子节点 3，随后新建了一个非叶子节点，并将 3 和 5 连接到了 上。



对于删除，我们考虑上面过程的逆过程。即找到与要删除的值权值相等的一个叶子节点，将它和它的父亲节点删除，并用其父亲的另一个儿子代替父亲的位置。

上面提到的建树也可以通过不断往树里插入节点实现，不过如果这样做必须要加入一个权值为 的节点作为根，否则会导致插入第一个元素的时候找不到大于自己的叶子节点。

代码实现：

```

1 /* 将某一节点的全部信息复制到另一节点上 */
2 void copynode(int x, int y) {
3     ls[x] = ls[y];
4     rs[x] = rs[y];
5     sz[x] = sz[y];
6     vl[x] = vl[y];
7 }
8
9 /* 判断某一节点是否为叶子节点 */
10 bool leaf(int x) { return !ls[x] || !rs[x]; }
11
12 void insert(int x, int v) {
13     if (leaf(x)) {
14         ls[x] = add(std::min(v, vl[x]));
15         rs[x] = add(std::max(v, vl[x]));
16         pushup(x);
17         maintain(x);
18         return;
19     }
20     if (vl[ls[x]] >= v) {
21         insert(ls[x], v);
22     } else {
23         insert(rs[x], v);
24     }
25     pushup(x);
26     maintain(x);
27 }
28
29 void delete(int x, int v, int fa) {
30     if (leaf(x)) {
31         if (ls[fa] == x) {
32             copynode(fa, rs[fa]);
33         } else {
34             copynode(fa, ls[fa]);
35         }
36         pushup(fa);
37         return;
38     }
39     if (vl[ls[x]] >= v) {
40         delete(ls[x], v, x);
41     } else {
42         delete(rs[x], v, x);
43     }
44     pushup(x);
45     maintain(x);
46 }

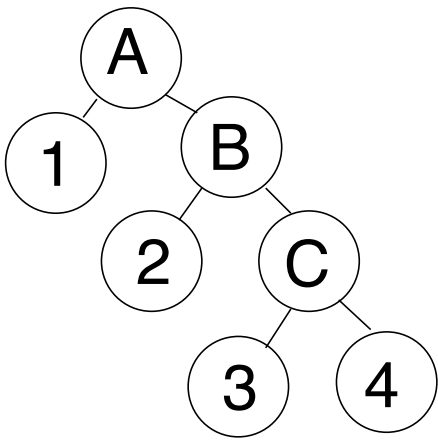
```

维护平衡

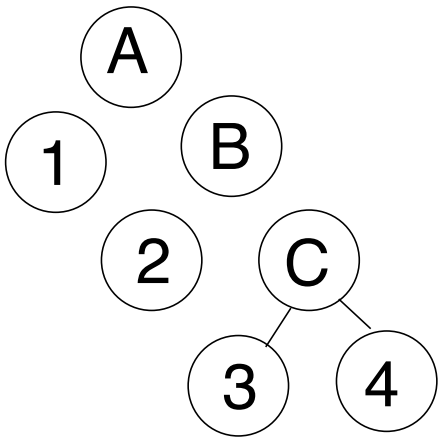
类似替罪羊树地，我们引入重构参数，我们设一个节点的平衡度为当前节点左子树的大小和节点大小的比值。当一个节点满足 $\frac{1}{2}$ 时，我们称其为 $\frac{1}{2}$ -平衡的。如果一棵 WBLT 的每一个节点都是 $\frac{1}{2}$ -平衡的，那么这棵树的树高一定能保证是量级的。证明是显然的，我们从一个叶子节点往父亲方向走，每次走到的节点维护的范围至少扩大到原来的 2 倍，那么树高就是量级的。

当某个节点不满足 $\frac{1}{2}$ -平衡时，说明这个节点是失衡的，我们需要重新维护平衡。但是和替罪羊树不同的是，WBLT 使用旋转操作维护平衡。旋转的大致过程为：将过重的儿子的两个儿子拆下来，一个和过轻的儿子合并，另一个成为一个新的儿子。

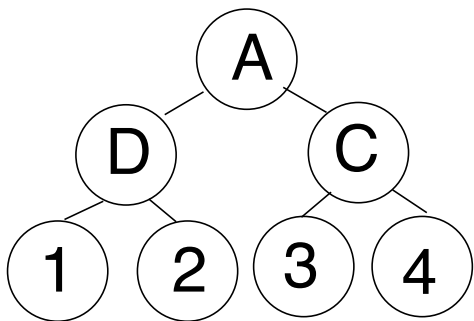
我们来举个例子：



这是一棵十分不平衡的 WBLT，节点 A 的右儿子显著地重于左儿子。我们先把右儿子及其两个儿子和左儿子都拆下来：

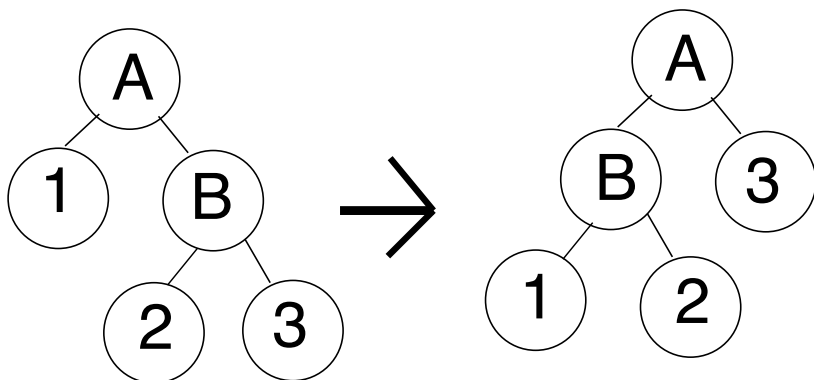


然后，我们将 B 和 C 两个节点合并作为 A 节点的左儿子，将 1 作为 A 的右儿子。由于 1 节点原本并不是叶子节点，因此其并不存储原始信息，直接删除就好。



旋转之后我们的树就变得十分平衡了。

但是上面的例子中，假设 A 节点的左子树过于大，我们把它合并到 A 的左子树上之后 A 的左子树又会很大，这时 A 依然可能不平衡。

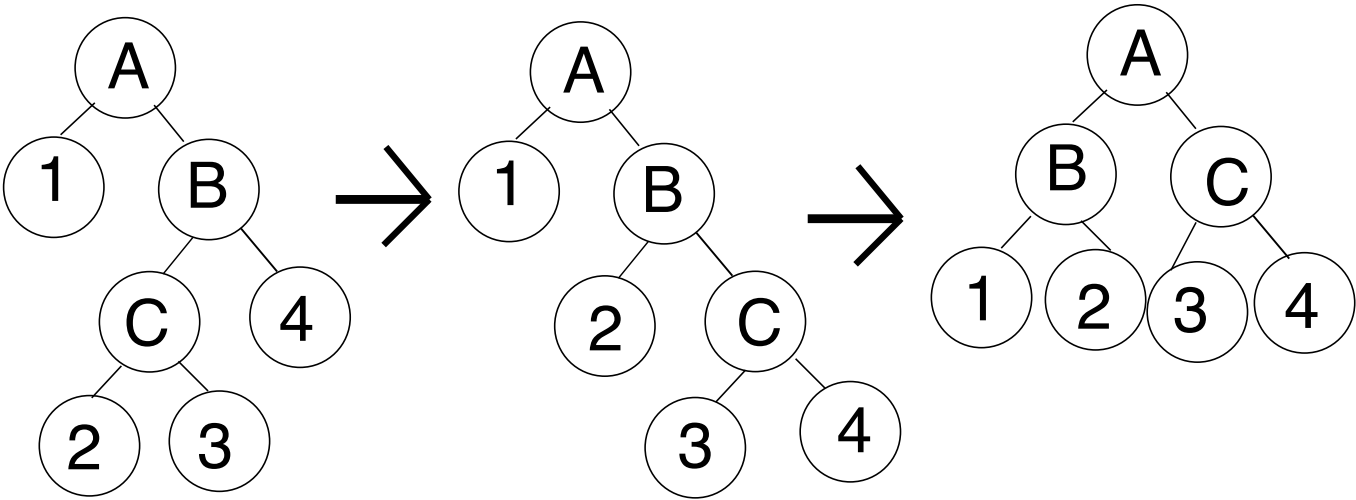


不失一般性，我们接下来仅讨论一个方向上的旋转，另一方向的旋转是对称的。我们不妨设 A 的平衡度为 B_A ， B 的平衡

度为 $BF(A)$ 。那么我们可以得到旋转后 A 的平衡度 $BF(A)$ ， B 的平衡度 $BF(B)$ ，推导过程直接将各节点大小用 $size$ 表示后代入定义式化简即可，这里略去。

不难发现仅当 $BF(A) \geq 1$ 且 $BF(B) \geq 1$ 时。

为了避免旋转后仍不平衡的情况出现，我们引入双旋操作。具体地，我们在较大子树上做一次相反方向的旋转操作，然后再维护当前节点的平衡。



类似地定义 $BF(B)$ ，则有 $BF(A) \geq 1$ 且 $BF(B) \geq 1$ 。可以证明当 $BF(A) \geq 1$ 且 $BF(B) \geq 1$ 时一定有 $BF(A) \geq 1$ 。

实现上，我们在 $BF(B) \geq 1$ 时进行单旋，否则进行双旋。

代码实现，这里取：

```

1 constexpr double alpha = 0.25;
2
3 int merge(int x, int y) {
4     int z = add(vl[x]);
5     ls[z] = x;
6     rs[z] = y;
7     pushup(z);
8     return z;
9 }
10
11 void rotate(int x, int flag) {
12     if (!flag) {
13         rs[x] = merge(rs[ls[x]], rs[x]);
14         ls[x] = ls[ls[x]];
15     } else {
16         ls[x] = merge(ls[x], ls[rs[x]]);
17         rs[x] = rs[rs[x]];
18     }
19 }
20
21 void maintain(int x) {
22     if (leaf(x)) return;
23     if (sz[ls[x]] > sz[rs[x]]) {
24         if (sz[rs[x]] >= sz[x] * alpha) return;
25         if (sz[rs[ls[x]]] >= sz[ls[x]] * (1 - alpha * 2) / (1 - alpha)) {
26             rotate(ls[x], 1);
27         }
28         rotate(x, 0);
29     } else {
30         if (sz[ls[x]] >= sz[x] * alpha) return;
31         if (sz[ls[rs[x]]] >= sz[rs[x]] * (1 - alpha * 2) / (1 - alpha)) {
32             rotate(rs[x], 0);
33         }
34         rotate(x, 1);
35     }
36 }

```

查询排名

我们发现 `WBLT` 的形态和线段树十分相似，因此查询排名可以使用类似线段树上二分的方式：如果左子树的最大值比大于等于待查值就往左儿子跳，否则就向右跳，同时答案加上左子树的 `size`。

```

1 int rank(int x, int v) {
2     if (leaf(x)) {
3         return 1;
4     }
5     if (vl[ls[x]] >= v) {
6         return rank(ls[x], v);
7     } else {
8         return rank(rs[x], v) + sz[ls[x]];
9     }
10 }

```

查询第 k 大的数

依然是利用线段树上二分的思想，只不过这里比较的是节点的大小。

```

1 int kth(int x, int v) {
2     if (sz[x] == v) {
3         return vl[x];
4     }
5     if (sz[ls[x]] >= v) {
6         return kth(ls[x], v);
7     } else {
8         return kth(rs[x], v - sz[ls[x]]);
9     }
10 }

```

合并两棵树

合并操作是指，给定两棵树，保证 α 的最大值不大于 α 的最小值，将它们合并为一棵大树，并且需要维护平衡。分情况讨论，不妨设：

- 若 x 为空树，则将 y 作为合并的结果。
- 若 $\alpha_x \geq \alpha_y$ ，说明可以以 x 为左右儿子建一个新树，则这棵新树的根节点满足 α -平衡，可以将这个新树作为结果。
- 否则若 $\alpha_x < \alpha_y$ ，则递归合并 x 的右儿子和 y ，再与 x 的左儿子合并。
- 否则将 x 的左儿子和 y 的右儿子的左儿子合并，将 x 的右儿子的右儿子与 y 合并，将两次合并的结果合并作为最终的结果。

不加证明地，这样合并的复杂度为 $O(n \log n)$ 。

代码实现：

```

1 int merges(int x, int y) {
2     if (!x || !y) return x + y;
3     if (min(sz[x], sz[y]) >= alpha * (sz[x] + sz[y])) {
4         return merge(x, y);
5     }
6     if (sz[x] >= sz[y]) {
7         if (sz[ls[x]] >= alpha * (sz[x] + sz[y])) {
8             return merges(ls[x], merges(rs[x], y));
9         } else {
10            return merges(merges(ls[x], ls[rs[x]]), merges(rs[rs[x]], y));
11        }
12    } else {
13        if (sz[rs[y]] >= alpha * (sz[x] + sz[y])) {
14            return merges(merges(x, ls[y]), rs[y]);
15        } else {
16            return merges(merges(x, ls[ls[y]]), merges(rs[ls[y]], rs[y]));
17        }
18    }
19 }

```

分裂

WBLT 的分裂与无旋 Treap 类似，根据子树大小或权值决定向下递归分裂左子树或右子树。不同的是，WBLT 需要对分裂出来的子树进行合并，以维护最终分裂的树的 α -平衡。

根据子树大小分裂的代码实现：


```

1 void split(int p, int k, int &x, int &y) {
2   if (!k) return x = 0, y = p, void();
3   if (leaf(p)) return x = p, y = 0, void();
4   if (k <= sz[ls[p]]) {
5     split(ls[p], k, x, y);
6     y = merges(y, rs[p]);
7   } else {
8     split(rs[p], k - sz[ls[p]], x, y);
9     x = merges(ls[p], x);
10  }
11 }

```

这样分裂的复杂度是 $O(k)$ 的，其中 n 是整棵树的大小。

总结

以上，我们利用 WBLT 完成了平衡树基本的几大操作。下面是用 WBLT 实现的 [普通平衡树模板](#)。

► 完整代码

因为 WBLT 可以合并与分裂，这意味着 WBLT 可以像无旋 Treap 一样实现插入删除等基本操作，写法同无旋 Treap。同理也可以实现文艺平衡树，下面是用 WBLT 实现的 [文艺平衡树模板](#)。

► 完整代码

注意 WBLT 需要两倍的空间；涉及分裂与合并时，需要注意垃圾回收，及时回收无用的节点，否则空间不是线性的。写法可参考文艺平衡树的代码。

本页面最近更新：2025/4/20 02:36:54，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[AtomAlpaca](#), [caijianhong](#), [cesonic](#), [H-J-Granger](#), [okumiii](#), [abc1763613206](#), [aofall](#), [c-forrest](#), [CoelacanthusHex](#), [countercurrent-time](#), [Enter-tainer](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [ksyx](#), [Marcythm](#), [NachtgeistW](#), [Persdre](#), [shuzhouliu](#), [stevebraveman](#), [SukkaW](#), [Tiphereth-A](#), [ziyao233](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用